

ASSIGNMENT 2: SYMMETRIC CRYPTO

Name	Student number
Chongkai (Lucas) Lyu	30349559
Leyang Gao	58256207

Questions 4–7 are answered in `attacker.py`; this PDF covers Questions 1–3.

1 Detecting Modes of Operation

[15 pts] Question 1 Which of `lyrics-ciphertext1.txt`, `lyrics-ciphertext2.txt`, and `lyrics-ciphertext3.txt` is AES-ECB, and how did you tell?

AES-ECB is: `lyrics-ciphertext3.txt`

Explanation: Each file stores Base64 text, not the raw ciphertext. I decoded it with `base64.b64decode(path.read_text().strip())`, then split the resulting bytes into 16-byte AES blocks.

`lyrics-ciphertext1.txt` has 181 blocks, all distinct. `lyrics-ciphertext2.txt` has 182 blocks, all distinct. `lyrics-ciphertext3.txt` has 182 blocks but only 171 distinct ones. Six block values repeat, including `95135c6db929844c03dec22524eddc6c` four times and `6b8273a4705485a6f43133c9061c643e` four times.

In ECB each block is encrypted on its own, so $M_i = M_j$ implies $C_i = C_j$. The lyrics repeat 16-byte phrases, and those repeats show up as identical ciphertext blocks only in `lyrics-ciphertext3.txt`. CBC and CTR do not preserve $M_i = M_j \Rightarrow C_i = C_j$, so that file is AES-ECB.

[15 pts] Question 2 Which one is AES-CTR, and how did you tell?

AES-CTR is: `lyrics-ciphertext1.txt`

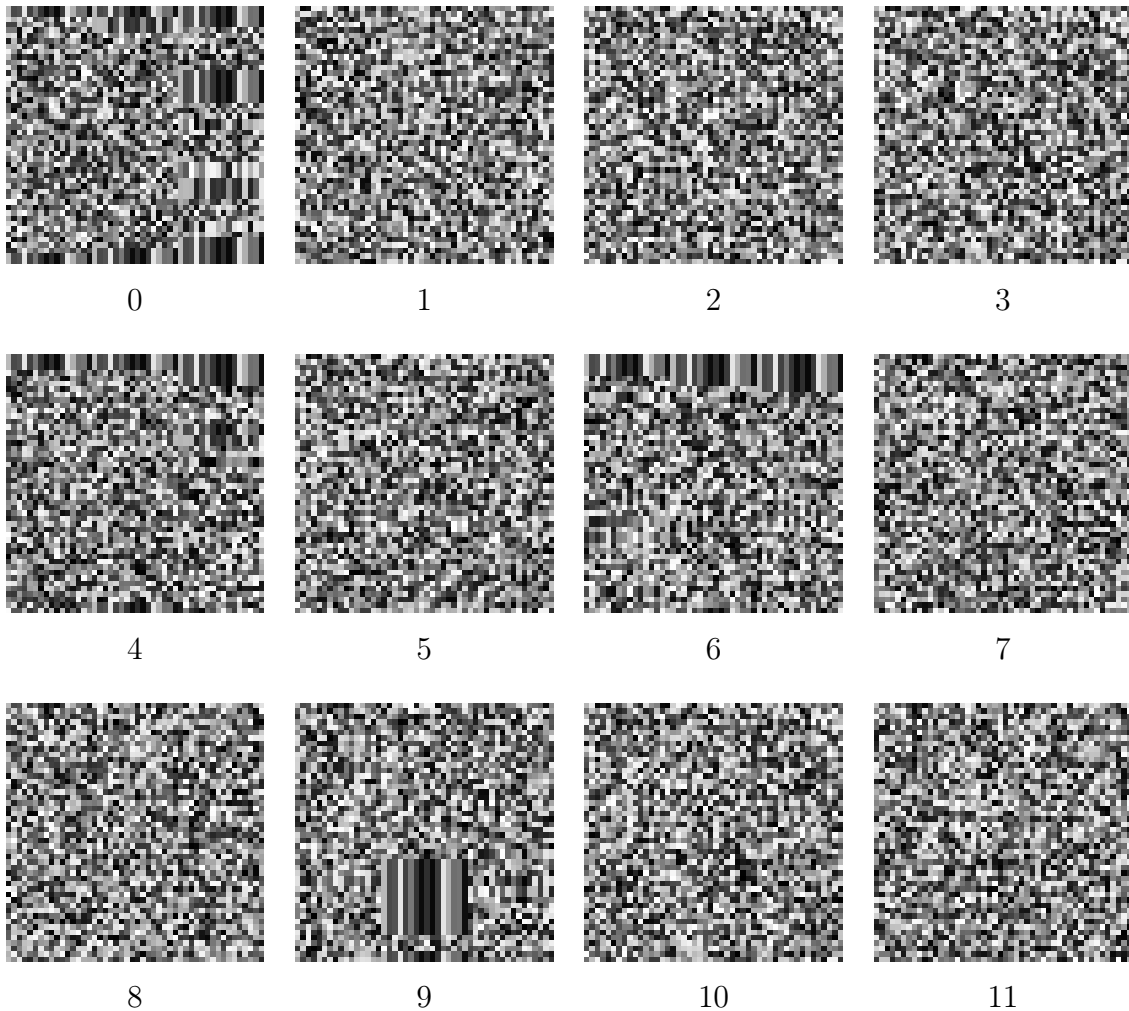
Explanation: The plaintext, `lyrics-plaintext.txt`, is 2896 bytes. That length is already a multiple of 16, but ECB and CBC still apply PKCS#7 padding: an aligned plaintext gets one extra block of sixteen `0x10` bytes before encryption. CTR does not pad, so its ciphertext stays the same length as the plaintext.

After the same Base64 decode, the lengths are:

lyrics-ciphertext1.txt	2896 bytes
lyrics-ciphertext2.txt	2912 bytes
lyrics-ciphertext3.txt	2912 bytes

Ciphertexts 2 and 3 are one block longer than the plaintext. Ciphertext 3 is the ECB file from Question 1, so the other long file is CBC. The short file matches the plaintext length and has no extra padding block, so lyrics-ciphertext1.txt is AES-CTR.

[30 pts] Question 3 Give the mode of operation of each image in ciphertext-sprites, and explain briefly how you found it.

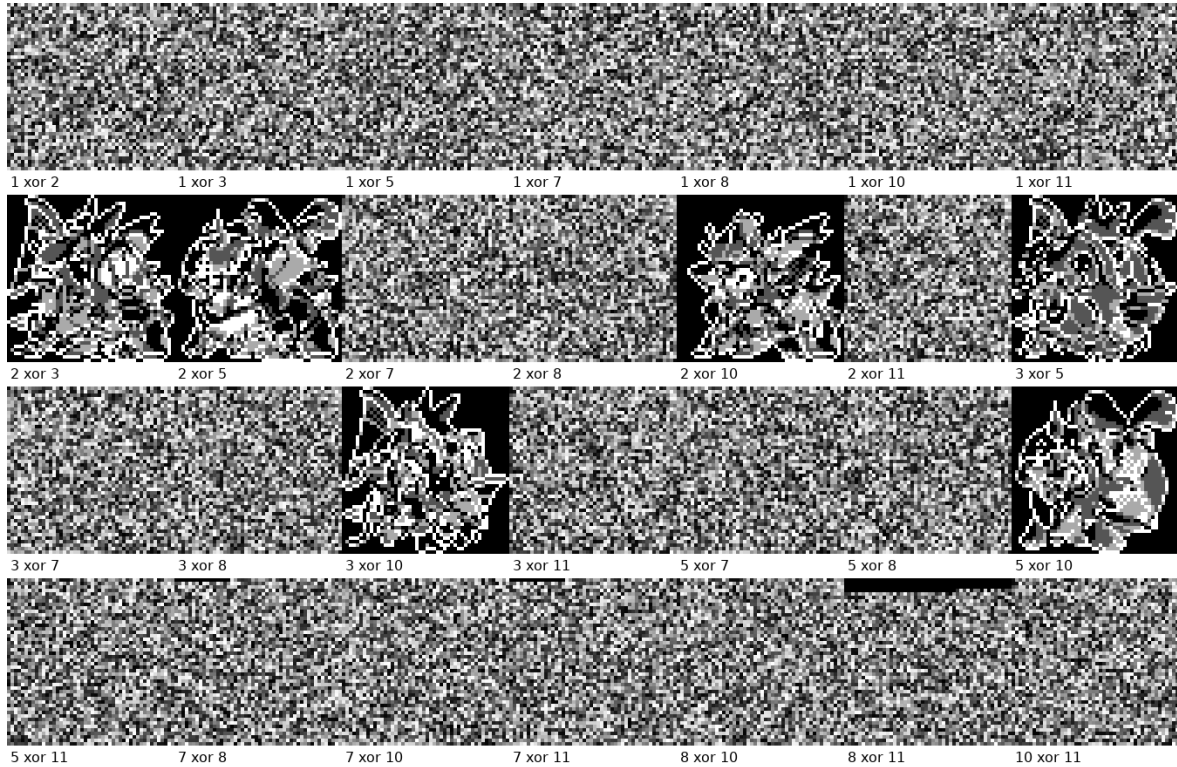


AES-ECB: 0, 4, 6, 9

Explanation: Images 0, 4, 6, and 9 still show repeated blocks of pixels. ECB encrypts each 16-byte block on its own, so $M_i = M_j$ implies $C_i = C_j$. A flat or repeated region of the original sprite stays visible after encryption. The other eight images look like noise and do not keep those repeated patches.

AES-CTR: 2, 3, 5, 10

Explanation: The other eight images were XORed in every pair. As shown below, CTR uses one shared key and an all-zero nonce, so those ciphertexts share a keystream. XORing two of them cancels that keystream and leaves the XOR of the two plaintexts, which still looks like a sprite. Every pair among 2, 3, 5, and 10 shows that shape, so those four are AES-CTR.



AES-CBC: 1, 7, 8, 11

Explanation: After removing the four ECB images and the four CTR images, 1, 7, 8, and 11 remain. Pairs that include one of these do not reveal a sprite, so they are AES-CBC.

Generative AI Disclosure